Fait le 18/12/2023

Travail réalisé par :

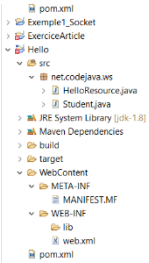
NDINGA-PAILLAT-MOPFOUTA Trésor Exaucé

Matière : SOA ET Service web

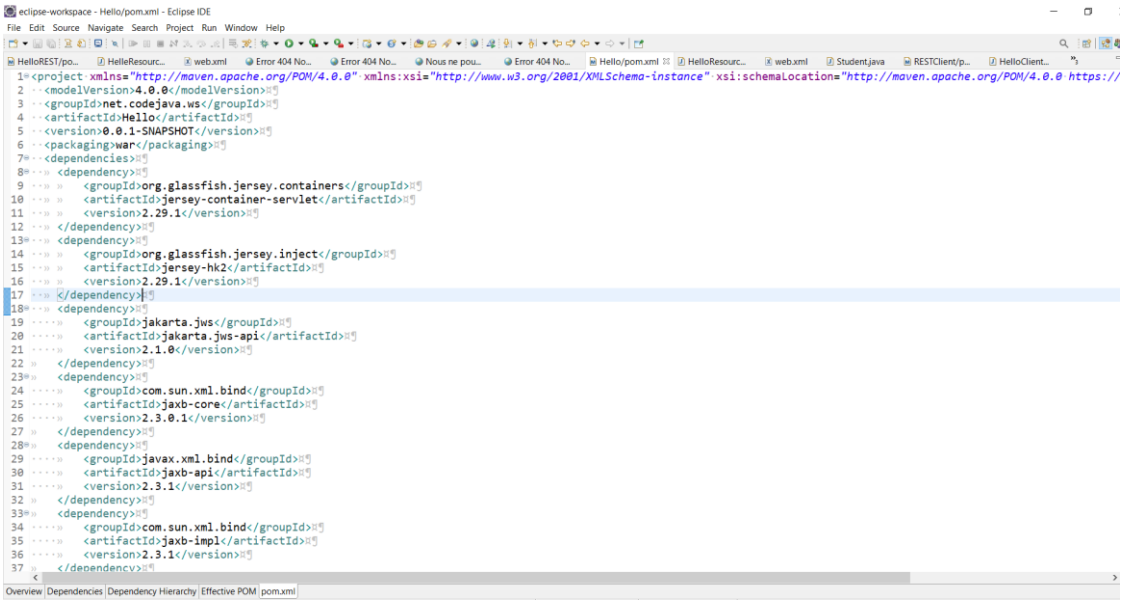
Prof : Mr Nissen MASMOUDI

RAPPORT DU TRAVAIL DE LA MISE EN PLACE D'UNE SOLUTION REST EN UTILSANT JERSEY

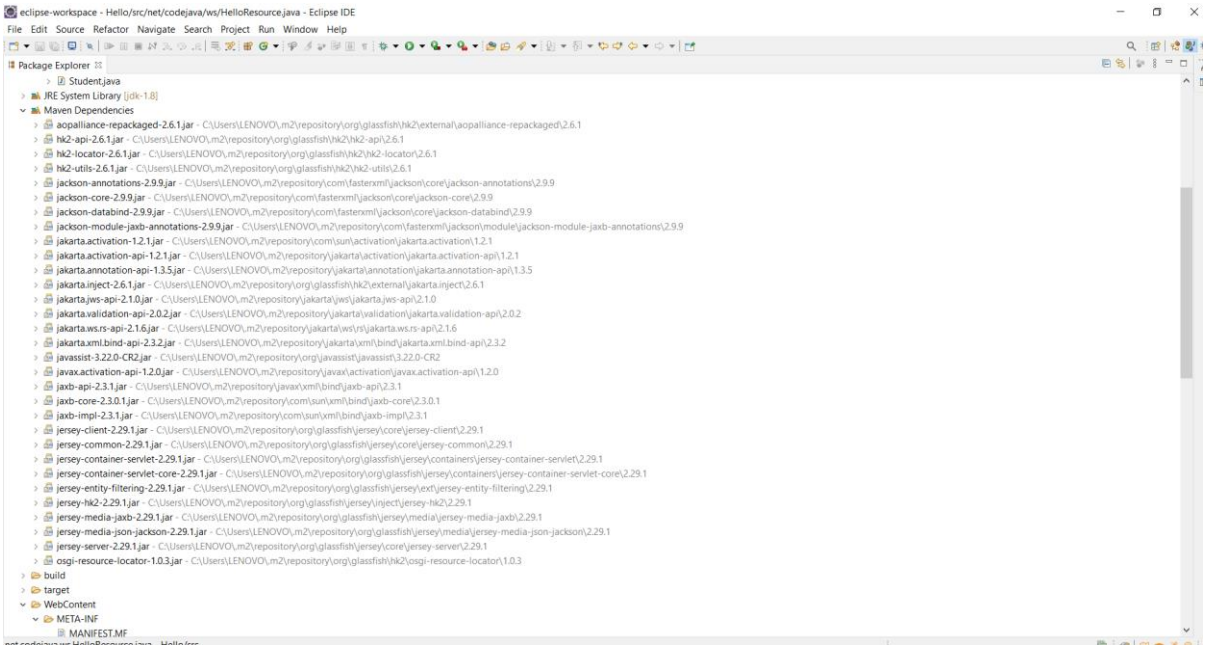
Comme il a été demandé, le but était de réussir à reproduire la vidéo en développant une solution REST Voici le travail que j'ai eu à faire premièrement créer le projet que j'ai nommé Hello ensuite le convertir en un projet Maven.



Après Ajouter des dépendances à mon projet notamment celles de Jersey dont on peut voir un extrait le voici



Y Compris les dépendances de Jackson faisant parties de Jersey voici un extrait



```
25      <artifactId>jaxb-core</artifactId>
26      <version>2.3.0.1</version>
27    </dependency>
28    <dependency>
29      <groupId>javax.xml.bind</groupId>
30      <artifactId>jaxb-api</artifactId>
31      <version>2.3.1</version>
32    </dependency>
33    <dependency>
34      <groupId>com.sun.xml.bind</groupId>
35      <artifactId>jaxb-impl</artifactId>
36      <version>2.3.1</version>
37    </dependency>
38    <dependency>
39      <groupId>org.glassfish.jersey.media</groupId>
40      <artifactId>jersey-media-json-jackson</artifactId>
41      <version>2.29.1</version>
42    </dependency>
43  </dependencies>
44  <build>
45    <sourceDirectory>src</sourceDirectory>
46    <plugins>
47      <plugin>
48        <artifactId>maven-compiler-plugin</artifactId>
49        <version>3.8.1</version>
50        <configuration>
51          <release>21</release>
52        </configuration>
53      </plugin>
54      <plugin>
55        <artifactId>maven-war-plugin</artifactId>
56        <version>3.2.3</version>
57        <configuration>
58          <warSourceDirectory>WebContent</warSourceDirectory>
59        </configuration>
60      </plugin>
```

Il a fallu modifier le fichier Web.xml en ajoutant des Servlet Pour assurer le Jersey REST Service on peut le voir sur l'écran ci-dessous comment ça s'affiche :

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://java.sun.com/xml/ns/javaee" xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://j
3   <display-name>Hello</display-name>
4   <welcome-file-list>
5     <welcome-file>index.html</welcome-file>
6     <welcome-file>index.htm</welcome-file>
7     <welcome-file>index.jsp</welcome-file>
8     <welcome-file>default.html</welcome-file>
9     <welcome-file>default.htm</welcome-file>
10    <welcome-file>default.jsp</welcome-file>
11  </welcome-file-list>
12  <servlet>
13    <servlet-name>Jersey Rest Service</servlet-name>
14    <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
15    <init-param>
16      <param-name>jersey.config.server.provider.packages</param-name>
17      <param-value>net.codejava.ws</param-value>
18    </init-param>
19  </servlet>
20  <servlet-mapping>
21    <servlet-name>Jersey Rest Service</servlet-name>
22    <url-pattern>/rest/*</url-pattern>
23  </servlet-mapping>
24
25
26 </web-app>
```

Le résultat des dépendances on obtient :

Créer un package nommer net.codejava.ws Dans le quel on a créé une classe appelée :

HelloResource dont on a importé les éléments suivants :

```
package net.codejava.ws;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
```

Suite à cela l'on a créer des méthodes à l'interieur de notre classe bien sûr notamment :

```
@GET
	@Produces(MediaType.TEXT_PLAIN)
	public String direBonjour() {
		return "Bonjour, tout le monde!";
	}
	@GET
	@Produces(MediaType.TEXT_HTML)
	public String sayHTMLHello()
	{
		return "<html><title>Hello REST</title><body>"
			+"<h1>Bonjour,tout le
monde!</h1></body></html>";
	}

	@GET
	@Produces(MediaType.APPLICATION_JSON)
	public Student sayJsonHello()
	{
		return new Student(123,"Trésor NDINGA","Congo");

	}
```

L'on peut voir l'extrait du Code dessous

eclipse-workspace - Hello/src/net/codejava/ws/HelloResource.java - Eclipse IDE

```java
package net.codejava.ws;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

@Path("bonjour")

public class HelloResource
{
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String direBonjour() {
        return "Bonjour, tout le monde!";
    }
    @GET
    @Produces(MediaType.TEXT_HTML)
    public String sayHTMLHello()
    {
        return "<html><title>Hello REST</title><body>"
                +"<h1>Bonjour,tout le monde!</h1></body></html>";
    }

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public Student sayJsonHello()
    {
        return new Student(123,"Trésor NDINGA","Congo");

    }


}
```

L’on peut voir l’exécution pour chacun d’entre eux sur le terminal et sur l’url

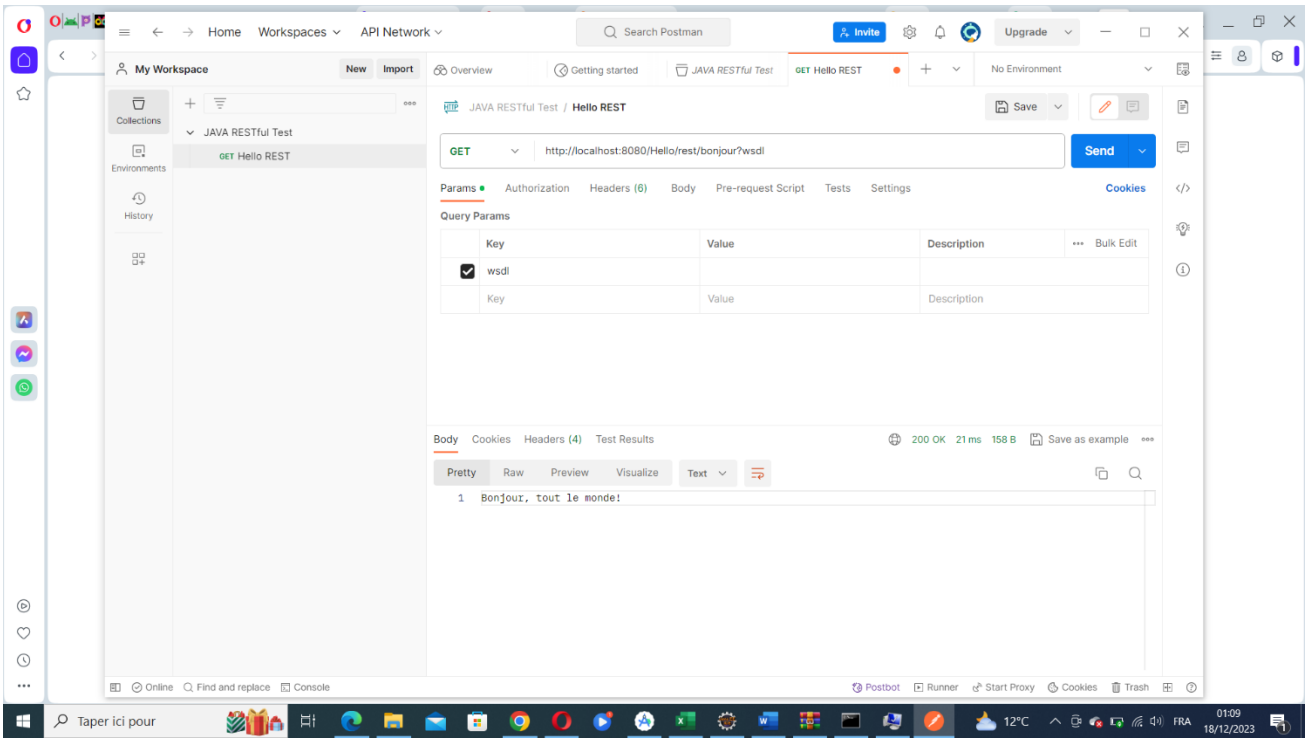
localhost:8080/Hello/rest/bonjour

Bonjour,tout le monde!

C:\Windows\system32\cmd.exe

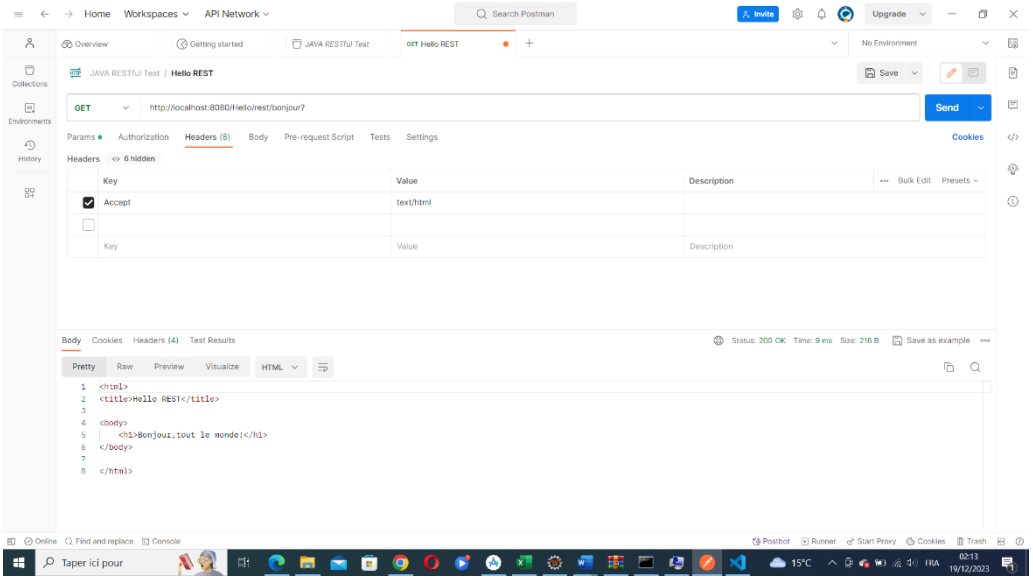
```
Microsoft Windows [version 10.0.19045.3803]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\LENOVO>curl
curl: try 'curl --help' for more information

C:\Users\LENOVO>curl http://localhost:8080/Hello/rest/bonjour?wsdl
Bonjour, tout le monde!
C:\Users\LENOVO>curl -v  http://localhost:8080/Hello/rest/bonjour?wsdl
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /Hello/rest/bonjour?wsdl HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Sun, 17 Dec 2023 23:46:23 GMT
< Content-type: text/plain
< Content-Length: 23
< Server: Jetty(9.4.35.v20201120)
<
Bonjour, tout le monde!* Connection #0 to host localhost left intact

C:\Users\LENOVO>curl -H "Accept: text/html"  http://localhost:8080/Hello/rest/bonjour?wsdl
<html><title>Hello REST</title><body><h1>Bonjour,tout le monde!</h1></body></html>
C:\Users\LENOVO>curl -H "Accept: application/json" http://localhost:8080/Hello/rest/bonjour?wsdl
{
"servlet":"Jersey Rest Service",
"message":"Not Acceptable",
"url":"/Hello/rest/bonjour",
"status":"406"
}
C:\Users\LENOVO>curl -H "Accept: application/json" http://localhost:8080/Hello/rest/bonjour
{
"servlet":"Jersey Rest Service",
"message":"Not Acceptable",
"url":"/Hello/rest/bonjour",
"status":"406"
}
C:\Users\LENOVO>curl -H "Accept: text/html"  http://localhost:8080/Hello/rest/bonjour?wsdl
<html><title>Hello REST</title><body><h1>Bonjour,tout le monde!</h1></body></html>
C:\Users\LENOVO>curl -H "Accept: application/json" http://localhost:8080/Hello/rest/bonjour
{"id":123,"name":"Trésor NDINGA","country":"Congo"}
C:\Users\LENOVO>
```

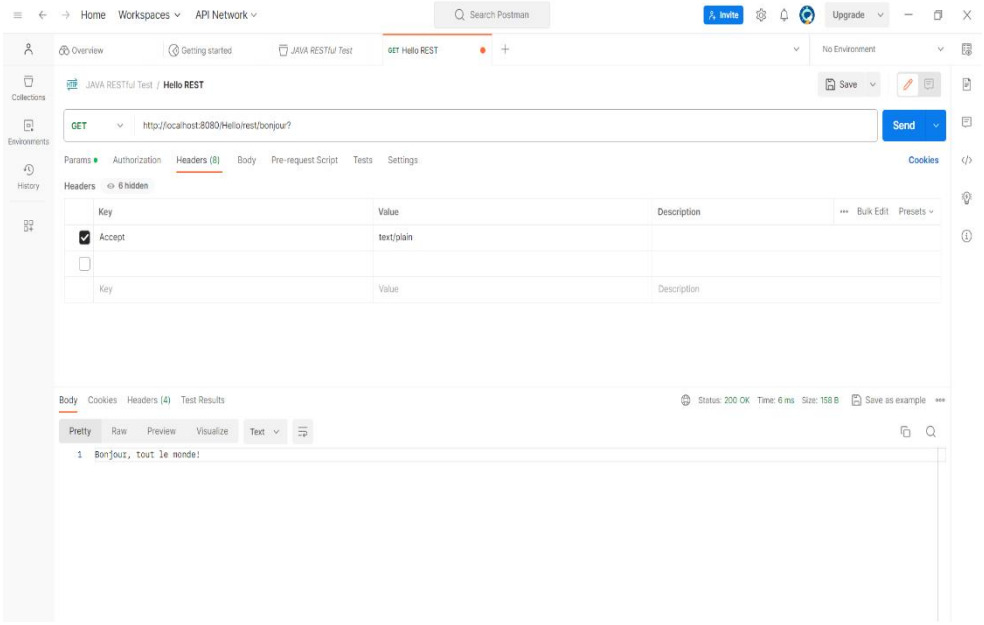
J'ai téléchargé POSTMAN afin de mieux voir et tester notre service rest en copiant le lien de l'URL et voici le résultat du premier test

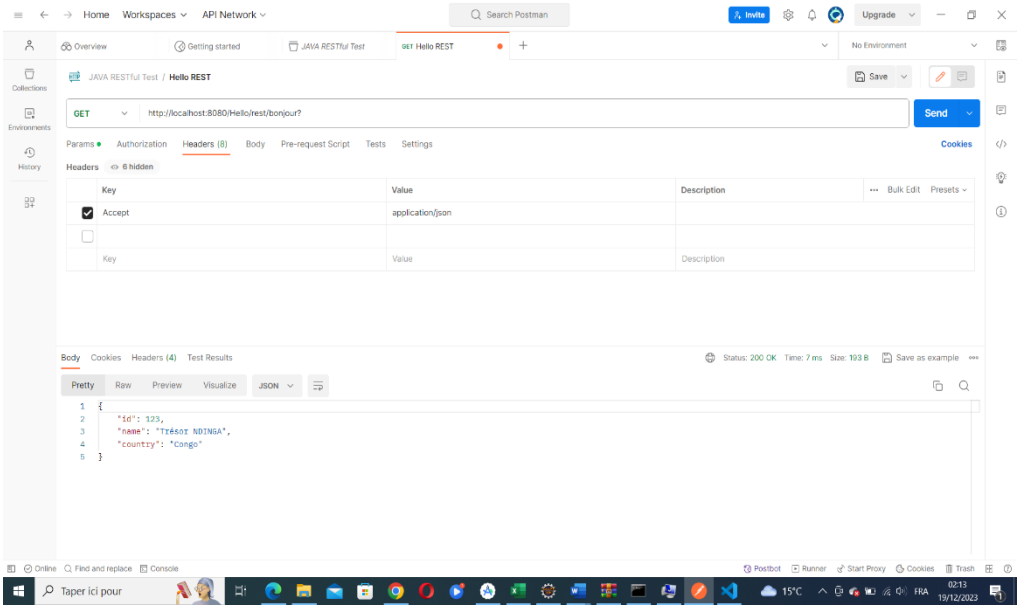


D'une manière résumée l'on peut constater que l'on a avancé dans le travail notamment sur :

- **Création du projet et dépendances spécifiées de Jersey**
- **Mise en place de la Classe Hello pour assurer le service RESTFUL**
- **On a configuré les servlet Jersey notamment Jersey Servlet Container**
- **Ensuite tester sur Chrome, Télécharger curl et le lancer via notre terminal et tester sur POSTMAN également.**







Sur les 3 images l'on peut voir que les différents tests ont été très bien éffectués celle de la

dernière image a été anticipée car il faut créer un nouveau projet Maven dans lequel on va créer le Client que l'on a nommé : RESTClient dans lequel on gardera le même nom de package soit net.codejava.ws que l'on va associer une classe soit :
Avec des importations de bibliothèques Java pour assurer un bon déroulement

```
Package net.codejava.ws ;
import javax.ws.rs.client.Client;
import javax.ws.rs.client.ClientBuilder;
import javax.ws.rs.client.WebTarget;
import javax.ws.rs.core.MediaType;

import org.glassfish.jersey.client.ClientConfig;




public class HelloClient {

	public static void main(String[] args) {
		// TODO Auto-generated method stub
		String uri="http://localhost:8080/Hello/rest/bonjour";
		ClientConfig config=new ClientConfig () ;
		Client client=ClientBuilder.newClient(config);
		WebTarget target =client.target(uri);
		String response=target.
request().accept(MediaType.APPLICATION_JSON).
get(String.class);

		System.out.println(response) ;
	}

}
```

```java
package net.codejava.ws;
import javax.ws.rs.client.Client;
import javax.ws.rs.client.ClientBuilder;
import javax.ws.rs.client.WebTarget;
import javax.ws.rs.core.MediaType;

import org.glassfish.jersey.client.ClientConfig;


public class HelloClient {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String uri="http://localhost:8080/Hello/rest/bonjour";
        ClientConfig config=new ClientConfig();
        Client client=ClientBuilder.newClient(config);
        WebTarget target =client.target(uri);
        String response=target.request().accept(MediaType.APPLICATION_JSON).get(String
            .class);

        System.out.println(response);
    }

}
```

Voici le résultat de l'exécution :

```
Console
<terminated> HelloClient [Java Application] C:\Program Files\Java\jdk-1.8\bin\javaw.exe (19 déc. 2023 à 02:28:55 – 02:28:56)
{"id":123,"name":"Trésor NDINGA","country":"Congo"}
```

Durant ce travail on a utilisé Tomcat 9.0 et le Server Local

```
Console  Servers
J2EE Preview at localhost [Started, Synchronized]
    Hello(Hello-0.0.1-SNAPSHOT) [Synchronized]
Tomcat v9.0 Server at localhost [Stopped, Synchronized]
    Hello(Hello-0.0.1-SNAPSHOT) [Synchronized]
```